

AEO Analysis: Railway Documentation

1. Current State (Strengths)

Railway has a mature AEO foundation: * **Content Negotiation:** `proxy.ts` auto-serves Markdown to 14+ AI crawlers. * **Corpus Dumps:** Dynamic `llms.txt` and `llms-full.txt`. * **JSON-LD Schema:** Auto-generates Organization, Breadcrumbs, Article, and FAQPage schemas.

2. Gap Analysis (By Domain)

🏗️ Infrastructure & Delivery

- **[P0] WAF Blocking AI Bots:** Cloudflare/Edge returns 403 for cloud ASNs (AWS, GCP). Major AI crawlers are blocked before reaching `proxy.ts`.
 - *Fix:* Audit WAF logs and propose "Verified Bot" rules or ASN whitelists to the core security team.
- **[P0] Fine-Grained Search Discovery:** Relying entirely on a wild-card `robots.txt` misses the opportunity to route specific bots.
 - *Fix:* Create a strict `robots.txt` directing known AI crawlers to the `/llms.txt` dumps, and implement an `agents.md` or `.well-known/ai-plugin.json`.
- **[Long-Term] Architectural Discussion: Fumadocs Core Migration:** The current `content-collections.ts` setup is heavy and requires manual `llms.txt` maintenance.
 - *Discussion:* We should evaluate migrating to Fumadocs core to natively automate MDX parsing.
 - *Pros:* Native page tree data generation, out-of-the-box `llms.txt` routing, eliminates 300+ `git log` sub-processes natively.
 - *Cons:* Heavy migration. Requires ripping out existing `proxy.ts` negotiation and rewriting React layout wrappers. Requires careful team discussion.

🧠 Content Architecture & Token Efficiency

- **[P1] AEO Token Efficiency Ratio (ATE):** The ultimate AEO metric is $\text{Efficiency} = \text{Answer-bearing tokens} / \text{Total tokens}$.
 - *Fix:* Integrate `js-tiktoken` into a CI Node script to programmatically measure the token ratio of high-traffic pages. Front-load CLI commands into the first 500 tokens.
- **[P1] Expand YAML Frontmatter Blocks:** The metadata block at the top of every `.md` file is crucial for automated search and retrieval.
 - *Fix:* Expand the frontmatter to require rich search metadata: explicit topics, Diátaxis intent, and **Last Modified Dates** so AI systems prioritize fresh content.
- **[P1] Unroll Hidden Instructions:** Critical commands are hidden behind `<Tabs>` and `<Accordion>` components.
 - *Fix:* Update MDX serializers to "unroll" interactive components into sequential markdown headings for AI endpoints.
- **[P1] Zod Schema Guardrails:** Without CI validation, developers forget to update the frontmatter block.
 - *Fix:* Enforce strict Zod schema validation on the frontmatter so the build fails if `lastUpdated` or `type: z.enum(['tutorial', 'how-to', 'reference', 'explanation'])` is missing.
- **[P1] Semantic HTML5:** HTML falls back to generic `<div>`s, confusing DOM scrapers.
 - *Fix:* Swap `<div className="docs-content">` to `<article>`.

🔧 Agent Tooling & Code

- **[P1] Model Context Protocol (MCP) Search:** Agents shouldn't have to ingest the full 300-page context dump.

- *Fix:* Expose Meilisearch as a Model Context Protocol (MCP) tool so Claude and local agents can dynamically query the docs natively via the Railway MCP server.
- **[P0] MDX Component Leaks:** The `.md` endpoint fails to resolve components like `<InstallCommand>`, leaking raw JSX.
 - *Fix:* Statically resolve critical MDX components into code blocks before serving to AI.
- **[Long-Term] Standalone App Examples:** Models and users need clean references.
 - *Fix:* Build standalone, highly-commented app templates to demonstrate capabilities and seed GitHub repos for LLM scraping.

🌟 Long-Tail & Corpus Seeding

- **[P2] Exact Error Strings:** Troubleshooting docs currently paraphrase errors.
 - *Fix:* Inject exact terminal error strings for 1:1 LLM matching.
- **[P2] Absolute URLs:** Root-relative links break when directly cited by LLMs.
 - *Fix:* Convert `/docs` to `https://docs.railway.com/docs` in markdown dumps.
- **[Long-Term] Actionable Quotes & Tool Comparisons:** LLMs need definitive consensus markers and evaluation-phase content.
 - *Fix:* Publish comparison articles (e.g., "Railway vs Heroku") and inject hard metrics ("*provisions in 10s*") and authoritative quotes.

📦 Migration & Documentation Scalability

- **[P1] AI-Assisted Migration with Zero-Touch Documentation:** The Vercel-to-Railway migration (via AI agents fed the docs site) uncovered a single critical documentation gap.
 - *Finding:* Railway's documentation correctly describes Nixpacks auto-detection of Astro projects and static site serving, but **fails to warn developers about removing Vercel-specific adapters** before deployment.
 - *Root Cause:* The docs lack a "Migration Checklist" section that consolidates platform-specific lock-in artifacts (adapters, analytics, config files) into a single, discoverable location.
 - *Fix:* Add dedicated migration guides following this structure:
 - **Pre-flight Checklist:** Framework-specific adapters to remove (e.g., `@astrojs/vercel`, `@vercel/analytics`)
 - **Dependency Alignment:** Node.js/runtime version requirements per framework version
 - **Build Configuration:** Critical files to delete or modify (`vercel.json`, `start` scripts)
 - **Runtime Binding:** Common "Application failed to respond" patterns and solutions
 - *Deliverables:*
 - `/docs/tutorials/migrate-from-vercel.mdx` (step-by-step, Diátaxis tutorial)
 - `/docs/reference/common-migration-errors.mdx` (lookup-oriented, error catalog)
 - Update frontmatter: `type: "tutorial", topics: ["deployment", "vercel", "migration"], lastUpdated: 2026-09-03`
 - *Outcome:* Replicable migration patterns reduce AI debugging loops. Future platform migrations (AWS → Railway, Netlify → Railway) will reference the same structure, making documentation self-scaling as the ecosystem grows.

30-Day AEO & SEO Execution Strategy

Objective: Establish Railway as the most cited cloud platform in generative search (ChatGPT, Claude, Perplexity) and dominate traditional SEO via high-leverage engineering and content automation.

Approach: High ownership, systems over coordination. We will use AI tooling (Claude Code) to execute bulk infrastructure changes, shifting human bandwidth to high-leverage writing and SoM benchmarking.

🟢 Week 1: Audit & Submit V1 Infrastructure PRs

Focus: Auditing the pipeline and working with core engineering to unblock edge delivery without breaking production.

- **Day 1-2: Audit & Submit CI Guardrail PRs**
 - Submit PR to enforce strict **Zod** schema validation for YAML Frontmatter (requiring topics, descriptions, Diátaxis tags, and **Last Modified Dates**).
 - Submit PR to add a Markdown link checker to CI.
- **Day 3-4: Propose Edge/WAF Unblocking Rules**
 - **Fix WAF:** Cloudflare/WAF currently returns 403 for cloud ASNs (AWS/GCP). Audit WAF logs and propose "Verified Bot" rules or ASN whitelists to the core security team.
 - **Action:** Stage a PR for a fine-grained `robots.txt` and `agents.md` explicitly routing known LLM crawlers to `.md` endpoints.
- **Day 5: Stage Deterministic AI Refactoring**
 - **Semantic HTML:** Stage PR swapping `<div className="docs-content">` for `<article>` in layouts.
 - **Absolute URLs:** Stage PR converting root-relative links to absolute URLs in `llms-full.txt.ts`.
 - **Performance:** Draft a script replacing `content-collections.ts` multi-process `git log` with a single-pass implementation.

🟡 Week 2: Semantic Signals & Dynamic Tooling

Focus: Equipping agents with explicit metadata and direct APIs.

- **Day 8-10: Automated Metadata Diátaxis**
 - Add type: `z.enum(['tutorial', 'how-to', 'reference', 'explanation'])` to `Zod`.
 - Use AI to infer and tag all 300+ docs. Update `seo.tsx` to inject schema based on tags.
- **Day 11-12: Expose Meilisearch via Model Context Protocol (MCP)**
 - **Action:** Expose Meilisearch as a Model Context Protocol (MCP) tool so Claude and local agents can dynamically query the docs index natively via the Railway MCP server, rather than relying on full context dumps.
- **Day 13-14: Unroll Hidden Instructions & Fix MDX Leaks**
 - Update `proxy.ts` to "unroll" `<Tabs>` and `<Accordion>` into sequential markdown headings.
 - Statically resolve components (like `<InstallCommand>`) into code blocks so they don't leak raw JSX into AI context.

🟠 Week 3: Content Engineering (ATE & Extraction)

Focus: Optimizing text for LLM attention spans and exact-match retrieval.

- **Day 15-17: Token Efficiency Optimization (ATE) via tiktoken**
 - **Action:** Write a Node.js script using `js-tiktoken` to programmatically calculate `Efficiency = Answer-bearing tokens / Total tokens`.
 - **Action:** Rewrite the top 20 highest-traffic pages to front-load summaries and CLI commands.
- **Day 18-19: Expand YAML Frontmatter Blocks**
 - **Action:** Expand the metadata block at the top of every markdown file. Inject **Last Modified Dates** and detailed topics. This guarantees LLMs recognize the freshness and scope of the documentation.
- **Day 20-21: Exact Error Strings & Density**
 - Inject literal terminal errors into troubleshooting docs to guarantee exact-match RAG retrieval.

- Break paragraphs >150 words with semantic H2/H3 headers.

🔴 Week 4: Documentation Gap Remediation & Proof of Concept

Focus: Converting AI migration findings into reusable docs, then testing structural splits.

- **Day 22-24: Convert Migration Report into Migration Guides**
 - **Action:** Extract insights from `railway-migration-report.md` and convert the 6 error categories into a reusable "Migration Runbook."
 - **Action:** Create two new docs:
 - `/docs/tutorials/migrate-from-vercel.mdx` — Step-by-step Diataxis tutorial with pre-flight checklists
 - `/docs/reference/common-migration-errors.mdx` — Lookup-oriented error catalog with solutions
 - **Content Structure:** Framework-specific adapters → Dependency alignment → Build config → Runtime binding
 - **Frontmatter:** Tag with type: "tutorial", topics: ["deployment", "vercel", "migration"], `lastUpdated: 2026-09-03`
 - **Outcome:** Future AI migrations reduce debugging loops. Docs become self-scaling as new platform migration patterns are added.
- **Day 25-27: Diátaxis Splitting (Proof of Concept)**
 - Break the monolithic `quick-start.md` into distinct, single-path tutorials (e.g., `tutorial-github.md`).
 - Unbury critical commands and verify all inbound links and redirects are updated cleanly.
- **Day 28-30: Establish Share of Model (SoM) Baselines**
 - Run 30 fixed category prompts across ChatGPT, Claude, and Perplexity (e.g., "What is the best way to deploy a Flask app?").
 - Track mention rate, accuracy, and URL citation rate to establish a baseline for your AEO growth.

🚀 Beyond 30 Days (Priority Initiatives)

High-impact projects to tackle once the baseline infrastructure and metrics are established.

- **[P1] Expand Standalone App Templates:** Build and maintain a growing library of highly-commented, production-ready app templates (e.g., "Deploying Django on Railway", "Next.js with PostgreSQL", "FastAPI with Workers").
 - *Rationale:* GitHub repos are heavily scraped by LLMs. Rich templates provide concrete examples, clarify capabilities, and directly seed high-quality training data for generative search. High ROI for extending existing templates.
- **[P1] Corpus Seeding (Tool Comparisons):** Publish comparative articles on external blogs and industry publications (e.g., "Railway vs Heroku", "Railway vs AWS Amplify"). Inject actionable quotes, quantifiable metrics, and first-hand performance data.
 - *Rationale:* Direct influence on LLM consensus building. Comparative content is a primary input for generative search answer formation. Critical for controlling narrative around platform selection decisions.
- **[Long-Term] Architectural Discussion: Fumadocs Core Migration:**
 - **Proposal:** Treat this as an Architecture Decision Record (ADR) to migrate off `content-collections.ts`.
 - **Pros:** Native page tree generation, faster MDX parsing, and out-of-the-box `llms.txt` routing without custom scripts.
 - **Cons:** High migration cost. Requires ripping out the existing `proxy.ts` negotiation and rewriting layout wrappers. Requires careful alignment with the core engineering team.
 - *Rationale:* Architectural debt with high complexity. Enables future scaling and maintenance efficiency, but not directly tied to immediate AEO growth. Requires core team alignment and should follow successful 30-day baseline establishment.